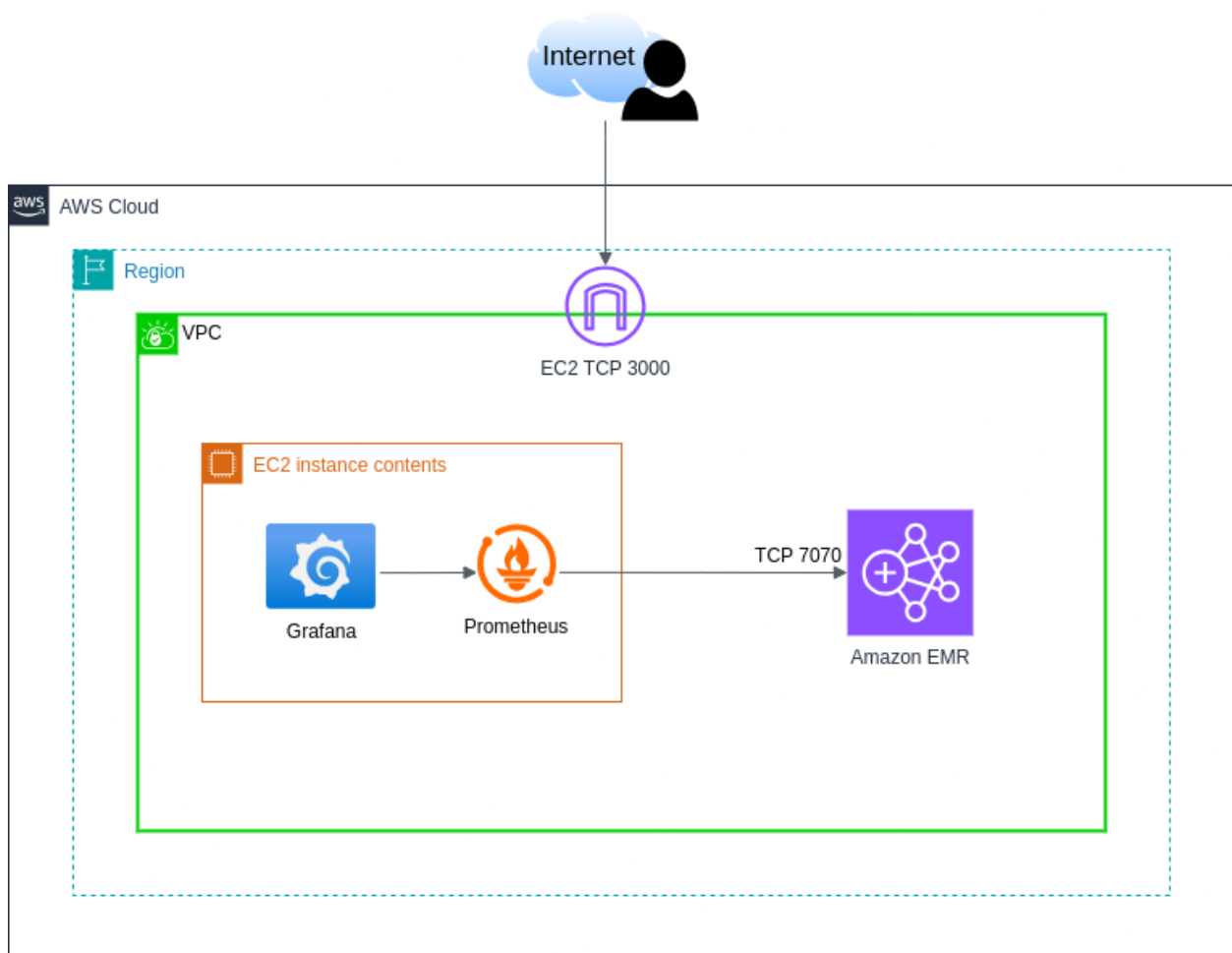


Tarea evaluable 3^a evaluación, Monitoreo en AWS EMR.

Autor : IABD05



Índice

1. Crear cluster EMR con script.....	3
Exposición del puerto 7070.....	3
Conexión por medio de SSH.....	3
Comprobación de la obtención de las métricas.....	6
2. Despliegue de Prometheus y Grafana en un EC2.....	7
3. Visualización de Métricas en Grafana.....	11
4. Resultado final del dashboard con las métricas.....	18

1. Configuración clúster EMR

1.1 Creación del clúster

```
/EMR_creator_n_checker/creator.py
Input the name for the cluster:
>> EMR-Monitoring-iabd05
Input the EMR version (example: 'emr-6.5.0')
>> emr-6.6.0
Input the apps to use, separate them with ','
>> hadoop,hive,spark,zeppelin
Input the number of slave instances (e.g., 2, 3):
>> 2
EMR cLuster created with ID: j-1PLWVHJ50AK6S
```

Figure 1: Se ha empleado un script en Python con la librería boto3

1.2 Exposición del puerto 7070

The screenshot shows the AWS IAM console 'Rules' page. A custom rule is defined with the following details:

- Rule type: Custom TCP
- Protocol: TCP
- Port: 7070
- Source: Anywh...
- Search: 0.0.0.0/0
- Buttons: Add rule, Delete

Figure 2: Sólomente para desarrollo, en producción habría que permitir únicamente el acceso a la IP de la instancia EC2 que va a realizar la consulta.

1.3 Conexión y configuración del nodo maestro

Deberemos conectarnos como root al nodo maestro, esto se realiza desde la consola de EC2.

```
Cluster status: j-1PLWVHJ50AK6S: WAITING
The cluster is ready
Master node public DNS: ec2-52-90-182-109.compute-1.amazonaws.com
```

Figure 3: Para conocer cuándo está el clúster listo, se emplea otro script.

☒	EMR-Monitoring-iabd05	i-0f51064773dcaed08	Running	m4.large	Initializing	View alarms +	us-east-1a	ec2-52-90-182-109.co...	52.90.182.109
☐	EMR-Monitoring-iabd05	i-0c194ca49f881a704	Running	m4.large	Initializing	View alarms +	us-east-1a	ec2-54-175-226-45.co...	54.175.226.45
☐	EMR-Monitoring-iabd05	i-01c17b7905dea5554	Running	m4.large	Initializing	View alarms +	us-east-1a	ec2-54-224-94-67.com...	54.224.94.67

Instance summary for i-04575fbeb8cecc03 (EMR-Monitoring-iabd05) [Info](#)

Updated less than a minute ago

[Connect](#) [Instance state](#) [Actions](#)

Instance summary for i-0f51064773dcaed08 (EMR-Monitoring-iabd05) [Info](#)

Updated less than a minute ago

[Connect](#) [Instance state](#) [Actions](#)

Note: In most cases, the default username, root, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.

[Cancel](#) [Connect](#)

Accedemos a la ruta /opt

[illegible]

Descargamos el jmx exporter

```
[root@ip-172-31-46-222 opt]# wget https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
--2025-03-29 13:28:03-- https://repo1.maven.org/maven2/io/prometheus/jmx/jmx_prometheus_javaagent/0.16.1/jmx_prometheus_javaagent-0.16.1.jar
Resolving repo1.maven.org (repo1.maven.org)... 146.75.36.209, 2a04:4e42:79::209
Connecting to repo1.maven.org (repo1.maven.org)|146.75.36.209|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 469645 (459K) [application/java-archive]
Saving to: 'jmx_prometheus_javaagent-0.16.1.jar'

100%[=====]
2025-03-29 13:28:03 (127 MB/s) - 'jmx_prometheus_javaagent-0.16.1.jar' saved [469645/469645]

[root@ip-172-31-46-222 opt]#
```

Creamos el archivo de configuración

```
[root@ip-172-31-46-222 opt]# nano config.yml
[root@ip-172-31-46-222 opt]# cat config.yml
lowercaseOutputName: true
lowercaseOutputLabelNames: true
rules:
- pattern: ".*"
[root@ip-172-31-46-222 opt]#
```

Figure 4: Con 'cat' se está mostrando el contenido del archivo 'config.yml'

Damos permisos al jar, al estar en un entorno de desarrollo no pasa nada por dar permisos de nivel 777, **pero esto en producción no se debe hacer ya que los permisos de nivel 777 son permisos absolutos para cualquiera que se haya conectado al clúster.**

```
[root@ip-172-31-46-222 opt]# ls
aws config.yml jmx_prometheus_javaagent-0.16.1.jar rh
[root@ip-172-31-46-222 opt]# chmod 777 jmx_prometheus_javaagent-0.16.1.jar
[root@ip-172-31-46-222 opt]# ls
aws config.yml jmx_prometheus_javaagent-0.16.1.jar rh
[root@ip-172-31-46-222 opt]#
```

Figure 5: LOS PERMISOS 777 NO SE DEBEN USAR EN ENTORNOS DE PRODUCCIÓN

Configuración del archivo de configuración de *Hadoop* para usar jmx exporter en el puerto 7070. En este caso nos interesa monitorear el *Namenode*, pero si se requiriese monitorear otros servicios de *Hadoop* (*Datanode*, *Resource Manager*, etc) se debería también indicar en el archivo de configuración de *Hadoop* `/etc/hadoop/conf/hadoop-env.sh`

```
export HADOOP_NAMENODE_OPTS=-javaagent:/opt/jmx_prometheus_javaagent-0.16.1.jar=7070:/opt/config.yml
```



Figure 6: Línea añadida al final del archivo `/etc/hadoop/conf/hadoop-env.sh`

Reiniciar el servicio

```
[root@ip-172-31-46-222 opt]# $ systemctl restart hadoop-namenode
-bash: $: command not found
[root@ip-172-31-46-222 opt]# $ systemctl restart hadoop-datanode
-bash: $: command not found
[root@ip-172-31-46-222 opt]# $ systemctl restart hadoop-resourcemanager
-bash: $: command not found
[root@ip-172-31-46-222 opt]# $ systemctl restart hadoop-nodemanager
```

En caso de que no funcione, reiniciamos el sistema

```
[root@ip-172-31-46-222 opt]# reboot
```

1.4 Comprobación de la obtención de las métricas

Para esto accederemos a la IP del nodo maestro por el puerto 7070, que es el que se ha expuesto

A screenshot of a web browser window. The address bar shows a "Not secure" warning and the URL "52.90.182.109:7070". The main content area displays a list of JMX metrics in a text-based format. Each metric entry starts with a comment line (e.g., "# HELP jmx_config_reload_failure_total") followed by a line indicating the metric type (e.g., "# TYPE jmx_config_reload_failure_total counter") and then the metric name and its value (e.g., "jmx_config_reload_failure_total 0.0"). The metrics cover JVM configuration, buffer pool usage, memory usage, and memory pool statistics.

```
# HELP jmx_config_reload_failure_total Number of times configuration have failed to be reloaded.
# TYPE jmx_config_reload_failure_total counter
jmx_config_reload_failure_total 0.0
# HELP jvm_buffer_pool_used_bytes Used bytes of a given JVM buffer pool.
# TYPE jvm_buffer_pool_used_bytes gauge
jvm_buffer_pool_used_bytes{pool="direct",} 1067692.0
jvm_buffer_pool_used_bytes{pool="mapped",} 0.0
# HELP jvm_buffer_pool_capacity_bytes Bytes capacity of a given JVM buffer pool.
# TYPE jvm_buffer_pool_capacity_bytes gauge
jvm_buffer_pool_capacity_bytes{pool="direct",} 1067692.0
jvm_buffer_pool_capacity_bytes{pool="mapped",} 0.0
# HELP jvm_buffer_pool_used_buffers Used buffers of a given JVM buffer pool.
# TYPE jvm_buffer_pool_used_buffers gauge
jvm_buffer_pool_used_buffers{pool="direct",} 25.0
jvm_buffer_pool_used_buffers{pool="mapped",} 0.0
# HELP jvm_memory_objects_pending_finalization The number of objects waiting in the finalizer queue.
# TYPE jvm_memory_objects_pending_finalization gauge
jvm_memory_objects_pending_finalization 0.0
# HELP jvm_memory_bytes_used Used bytes of a given JVM memory area.
# TYPE jvm_memory_bytes_used gauge
jvm_memory_bytes_used{area="heap",} 5.880928E7
jvm_memory_bytes_used{area="nonheap",} 5.3453552E7
# HELP jvm_memory_bytes_committed Committed (bytes) of a given JVM memory area.
# TYPE jvm_memory_bytes_committed gauge
jvm_memory_bytes_committed{area="heap",} 2.48512512E8
jvm_memory_bytes_committed{area="nonheap",} 5.4943744E7
# HELP jvm_memory_bytes_max Max (bytes) of a given JVM memory area.
# TYPE jvm_memory_bytes_max gauge
jvm_memory_bytes_max{area="heap",} 9.54728448E8
jvm_memory_bytes_max{area="nonheap",} -1.0
# HELP jvm_memory_bytes_init Initial bytes of a given JVM memory area.
# TYPE jvm_memory_bytes_init gauge
jvm_memory_bytes_init{area="heap",} 1.32120576E8
jvm_memory_bytes_init{area="nonheap",} 2555904.0
# HELP jvm_memory_pool_bytes_used Used bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_used gauge
jvm_memory_pool_bytes_used{pool="Code Cache",} 6534848.0
jvm_memory_pool_bytes_used{pool="Metaspace",} 4.20596E7
jvm_memory_pool_bytes_used{pool="Compressed Class Space",} 4859104.0
jvm_memory_pool_bytes_used{pool="PS Eden Space",} 2.0271928E7
jvm_memory_pool_bytes_used{pool="PS Survivor Space",} 8991248.0
jvm_memory_pool_bytes_used{pool="PS Old Gen",} 2.9546104E7
# HELP jvm_memory_pool_bytes_committed Committed bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_committed gauge
jvm_memory_pool_bytes_committed{pool="Code Cache",} 6619136.0
jvm_memory_pool_bytes_committed{pool="Metaspace",} 4.3163648E7
jvm_memory_pool_bytes_committed{pool="Compressed Class Space",} 5160960.0
jvm_memory_pool_bytes_committed{pool="PS Eden Space",} 1.4155776E8
jvm_memory_pool_bytes_committed{pool="PS Survivor Space",} 9437184.0
jvm_memory_pool_bytes_committed{pool="PS Old Gen",} 9.7517568E7
# HELP jvm_memory_pool_bytes_max Max bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_max gauge
jvm_memory_pool_bytes_max{pool="Code Cache",} 2.5165824E8
jvm_memory_pool_bytes_max{pool="Metaspace",} -1.0
jvm_memory_pool_bytes_max{pool="Compressed Class Space",} 1.073741824E9
jvm_memory_pool_bytes_max{pool="PS Eden Space",} 3.30825728E8
jvm_memory_pool_bytes_max{pool="PS Survivor Space",} 9437184.0
jvm_memory_pool_bytes_max{pool="PS Old Gen",} 7.16177408E8
# HELP jvm_memory_pool_bytes_init Initial bytes of a given JVM memory pool.
# TYPE jvm_memory_pool_bytes_init gauge
jvm_memory_pool_bytes_init{pool="Code Cache",} 2555904.0
jvm_memory_pool_bytes_init{pool="Metaspace",} 0.0
jvm_memory_pool_bytes_init{pool="Compressed Class Space",} 0.0
jvm_memory_pool_bytes_init{pool="PS Eden Space",} 3.3554432E7
```

Figure 7: Tras esto, una vez llevado a producción, habría que permitir únicamente el acceso a esta ip por este puerto a la IP del EC2 que va a monitorear el EMR

2. Despliegue de Prometheus y Grafana en un EC2

2.1 Crear instancia EC2 con script

```
Instance details
  Name: Prom-Graf-EC2
  ID: i-000a31e013c656354
  IP: 18.209.177.138
```

La VPC debe ser la misma que la instancia EMR, como solo tenemos 1 creada en AWS, no hay posibilidad de fallo

2.2 Descarga de Prometheus y configuración

Nos conectamos por ssh e instalamos ambos servicios

```
ssh -i labsuser.pem ubuntu@18.209.177.138
The authenticity of host '18.209.177.138 (18.209.177.138)' can't be established.
ED25519 key fingerprint is SHA256:hTiSNSzFQoHzapaLSJHFLtJpkr6LbcEkIGKUNJEJfr4.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '18.209.177.138' (ED25519) to the list of known hosts
.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.8.0-1021-aws x86_64)
```

```
ubuntu@ip-172-31-39-238:~$ wget https://github.com/prometheus/prometheus/releases/download/v2.30.3/prometheus-2.30.3.linux-amd64.tar.gz
```

Figure 8: Descarga de Prometheus

```
ubuntu@ip-172-31-39-238:~$ ls
prometheus-2.30.3.linux-amd64.tar.gz
ubuntu@ip-172-31-39-238:~$ 
ubuntu@ip-172-31-39-238:~$ tar -xzf prometheus-2.30.3.linux-amd64.tar.gz
ubuntu@ip-172-31-39-238:~$ ls
prometheus-2.30.3.linux-amd64  prometheus-2.30.3.linux-amd64.tar.gz
ubuntu@ip-172-31-39-238:~$
```

Figure 10: Descompresión

Ahora hay que modificar el archivo de configuración para que tenga como objetivo el clúster EMR, al final del documento agregaremos:

```
ubuntu@ip-172-31-39-238:~$ cd prometheus-2.30.3.linux-amd64/
ubuntu@ip-172-31-39-238:~/prometheus-2.30.3.linux-amd64$ nano prometheus.yml
```

```
GNU nano 7.2                                prometheus.yml
# A scrape configuration containing exactly one endpoint to scrape:
# Here it's Prometheus itself.
scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped
  - job_name: "prometheus"

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ["localhost:9090"]

  - job_name: 'emr-namenode'
    static_configs:
      - targets: ['172.31.46.222:7070']

[ Wrote 33 lines ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

Figure 11: Indicamos un nuevo trabajo, con un objetivo, la ip del EC2 maestro por el puerto expuesto

Ejecución de Prometheus con nohup, esto permite seguir interactuando con la consola y que el servicio permanezca encendido una vez cerrada la conexión ssh

```
ubuntu@ip-172-31-39-238:~/prometheus-2.30.3.linux-amd64$ nohup ./prometheus --co
nfig.file=prometheus.yml &
[1] 2205
ubuntu@ip-172-31-39-238:~/prometheus-2.30.3.linux-amd64$ nohup: ignoring input a
nd appending output to 'nohup.out'

ubuntu@ip-172-31-39-238:~/prometheus-2.30.3.linux-amd64$
```

Figure 12: La salida de la ejecución será almacenada en el fichero 'nohup.out'

2.3 Instalación de Grafana

```
ubuntu@ip-172-31-39-238:~/prometheus-2.30.3.linux-amd64$ sudo apt-get install -y
apt-transport-https
sudo apt-get install -y software-properties-common wget
wget -q -O - https://packages.grafana.com/gpg.key | sudo apt-key add -
echo "deb https://packages.grafana.com/oss/deb stable main" | sudo tee -a /etc/a
pt/sources.list.d/grafana.list
sudo apt-get update
sudo apt-get install -y grafana
sudo systemctl start grafana-server
sudo systemctl enable grafana-server
```

Figure 13: Instalación de Grafana

2.4 Acceso y configuración de Grafana

Para configurar Grafana accederemos a la ip del EC2 por el puerto 3000, que es el de grafana
Y para ello deberemos permitir la conexión por ese puerto en el grupo de seguridad

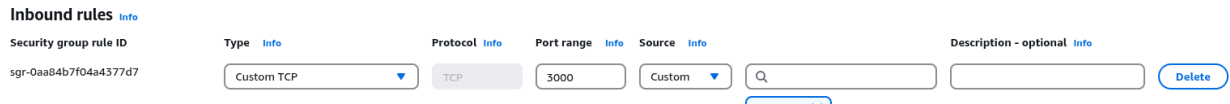


Figure 14: En producción solo se permitiría el acceso al puerto 3000 las IPs destinadas al uso de esta instancia

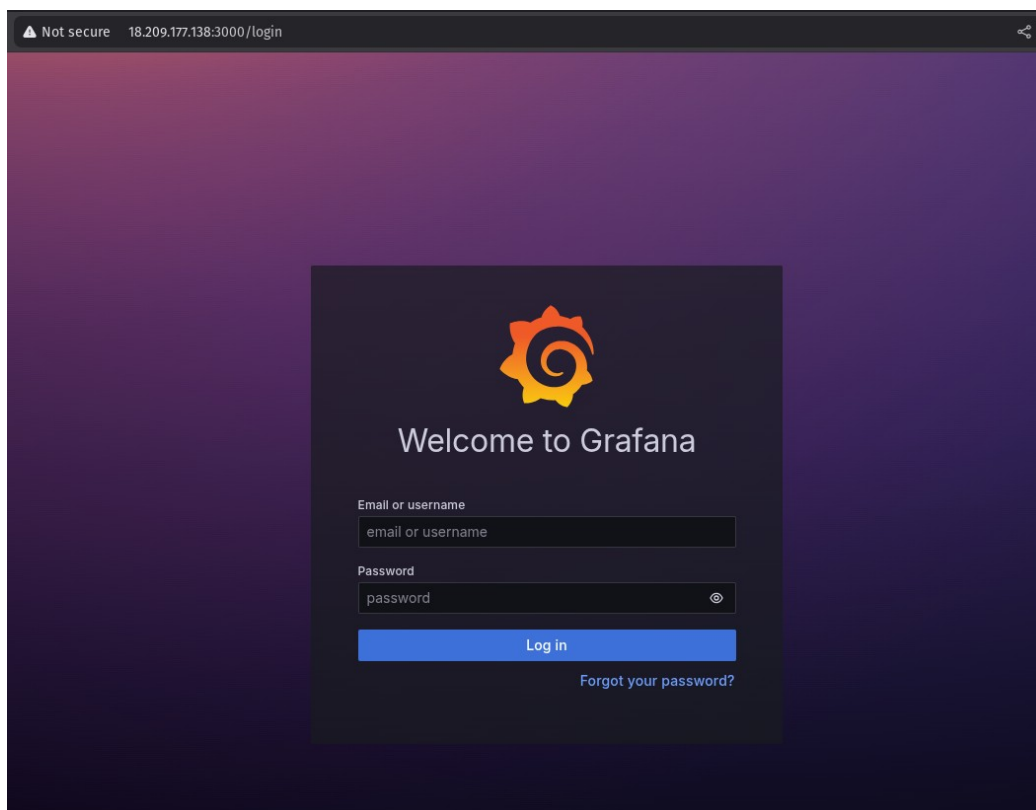
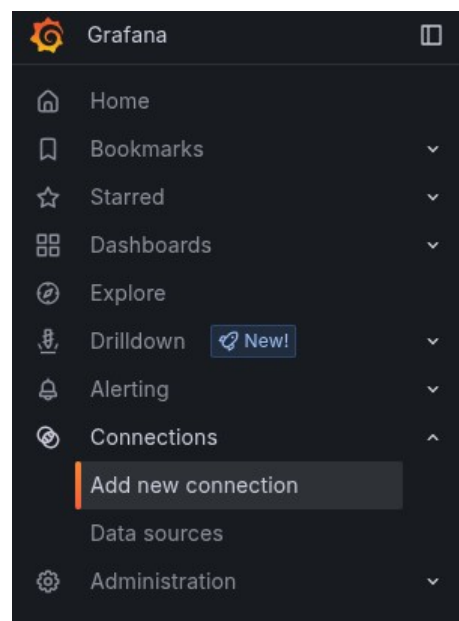
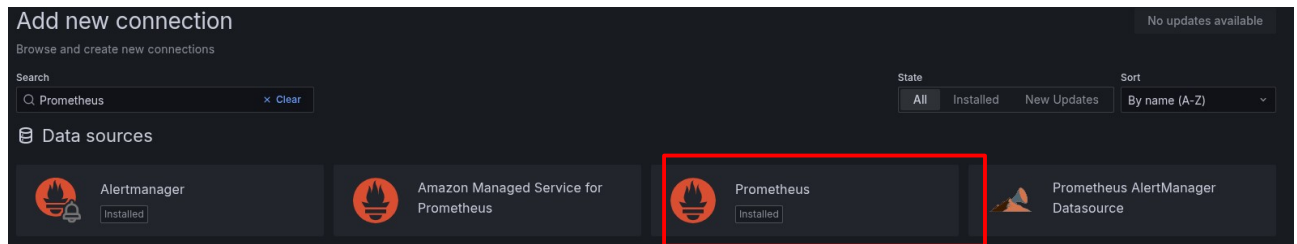


Figure 15: Iniciamos sesión con 'admin' - 'admin'

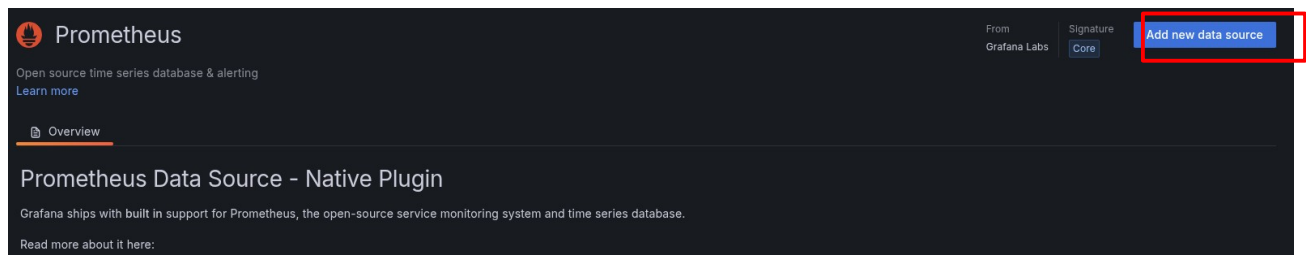
En el menú lateral accedereos a *Connections*
> *Add new connection*



De todos los que aparecen seleccionaremos Prometheus

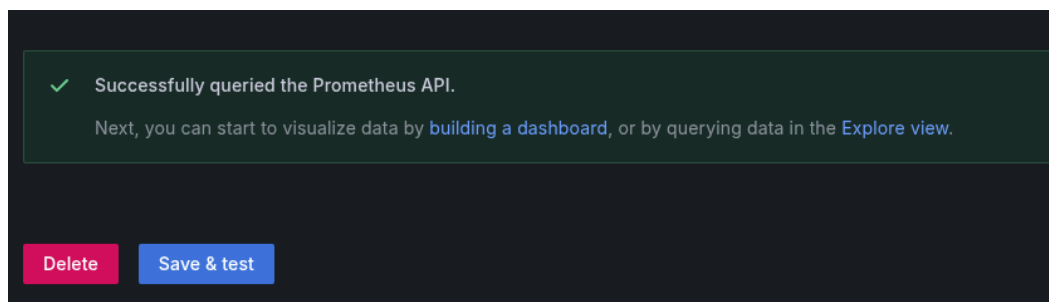
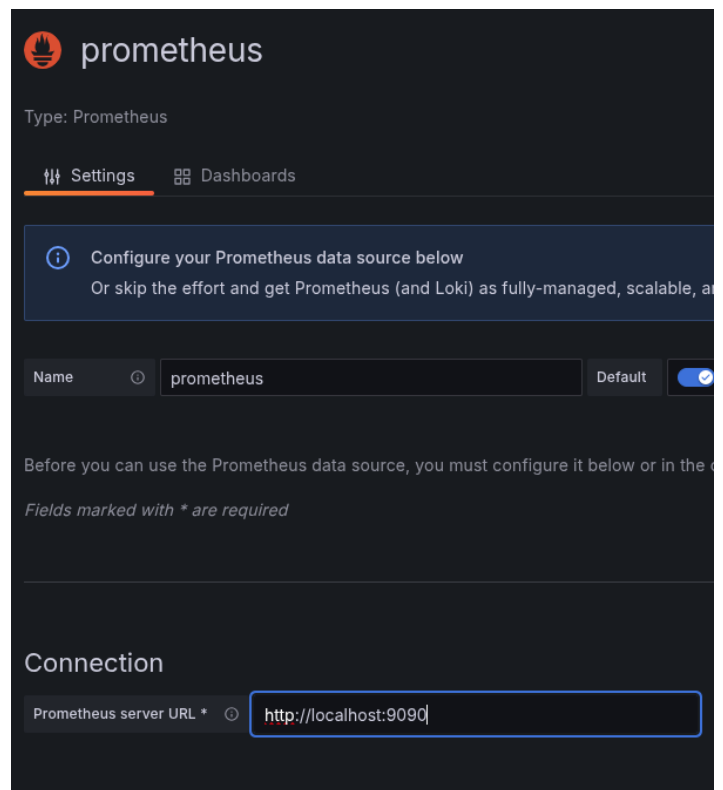


Y una vez seleccionado deberemos confirmar seleccionando 'Add new data source'



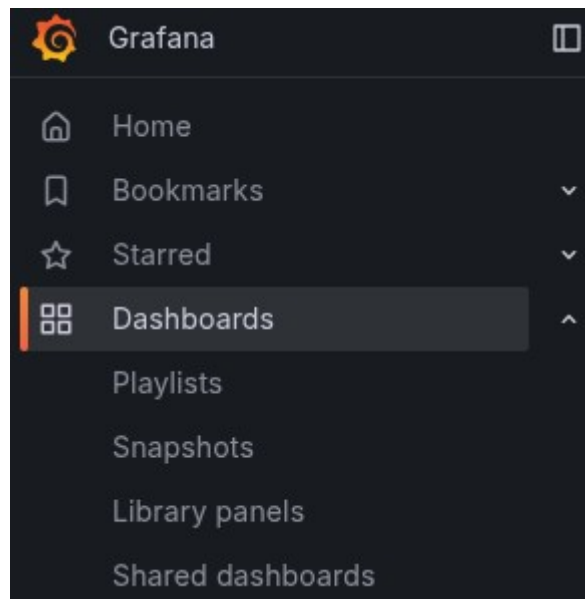
En el apartado de 'Connection' indicaremos <http://localhost:9090> que es la dirección donde se está ejecutando Prometheus y su puerto por defecto.

Tras esto deberemos confirmar abajo del todo con el botón 'save & test'; esto nos debe mostrar el mensaje en verde.

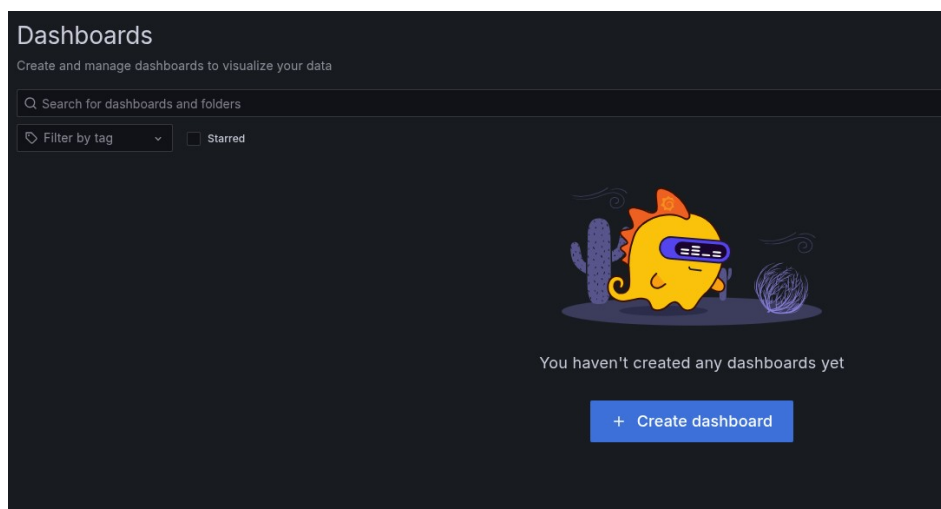


3. Visualización de Métricas en Grafana

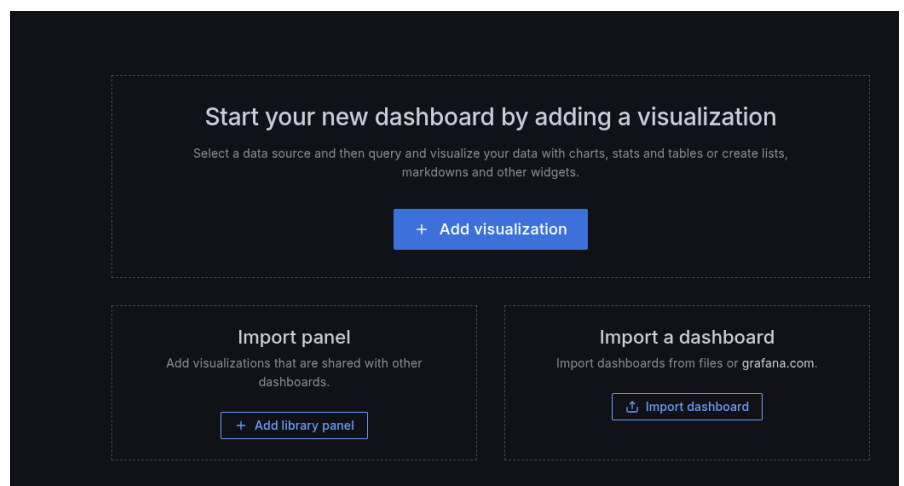
Para la visualización de las métricas, deberemos crear un dashboard; esto se consigue desde el menú lateral ‘Dashboards’.



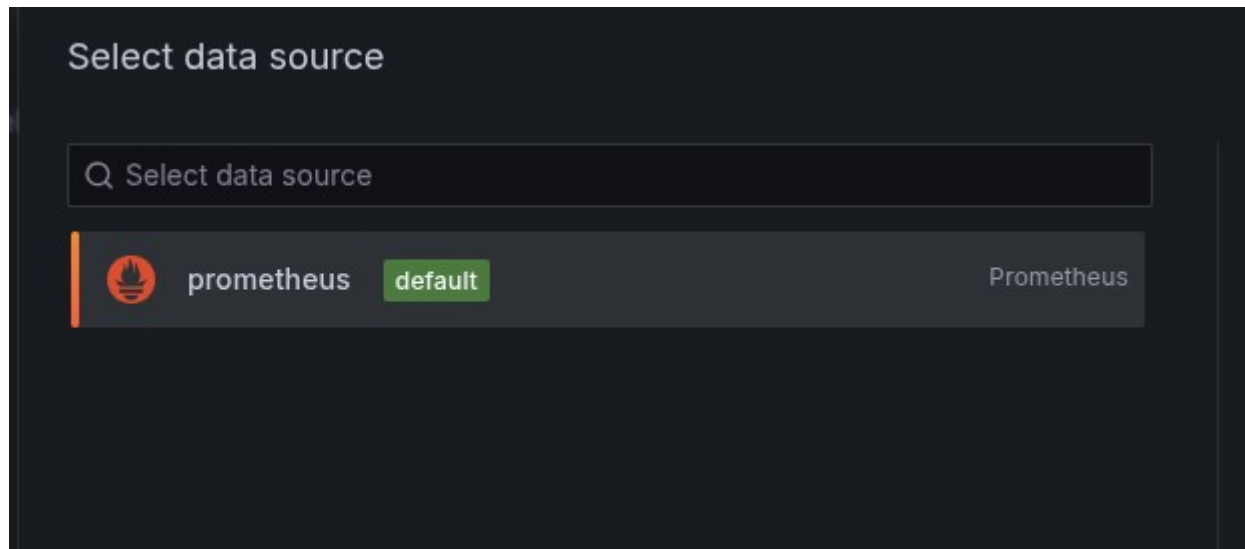
Una vez dentro indicaremos que queremos crear un dashboard.



Y añadir una visualización



Seleccionando como recurso a *Prometheus*



E introducimos la métrica que queremos consultar, podremos indicar que queremos métricas como

jvm_memory_bytes_used

Para obtener solo en el área heap, o podremos indicar solamente

jvm_memory_bytes_used

Para que se muestre algo como esto



Para ello realizaremos la siguiente consulta desde una terminal

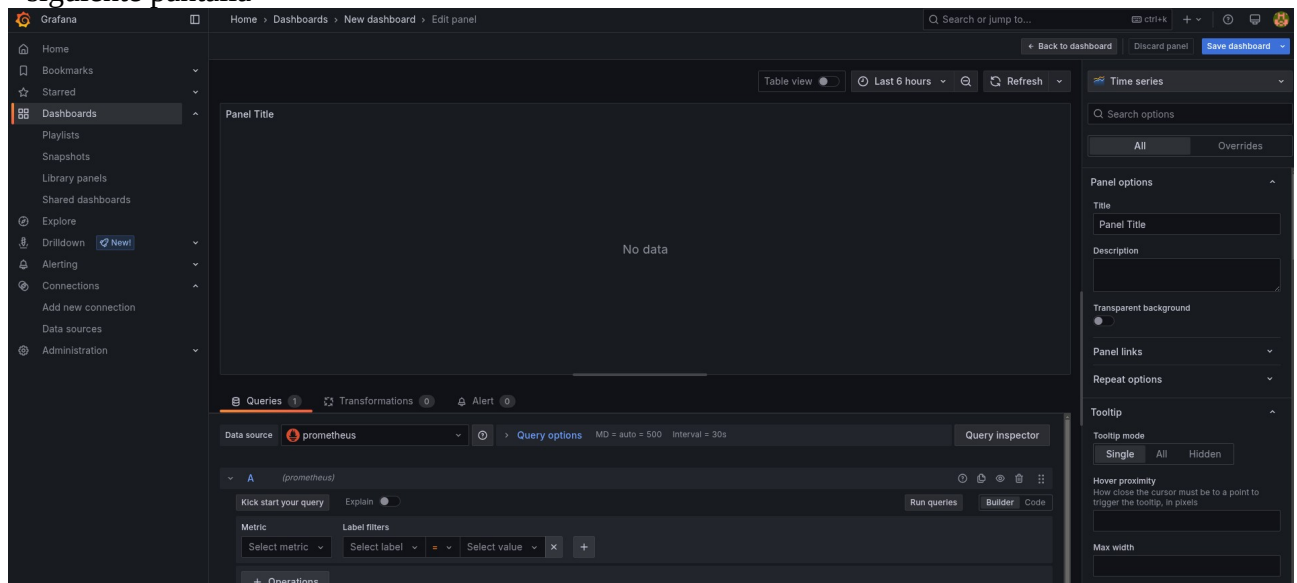
Esto nos devolverá todas las métricas filtradas por la palabra clave `'cpu'`

```
% Total      % Received % Xferd      Average Speed      Time      Time      Time      Current
   0      0      0      0      0      Dload  Upload    Total     Spent    Left     Speed

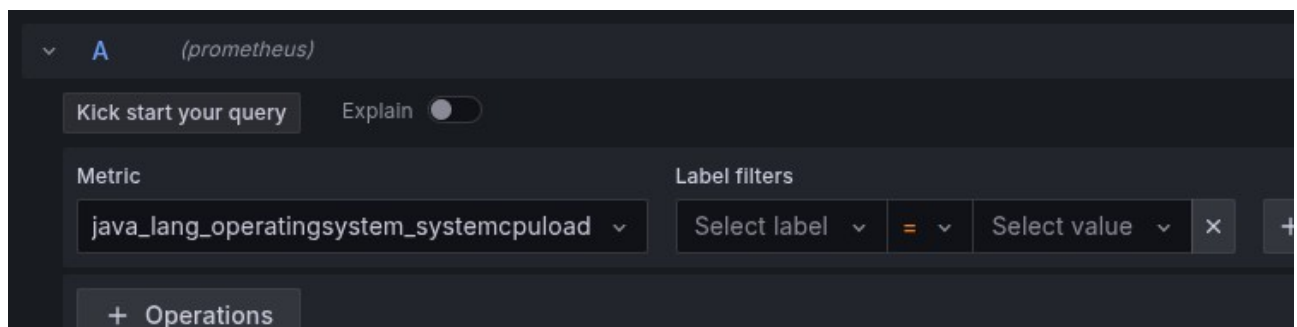
   0      0      0      0      0      0      0      0      0      0      0      0      0# HELP java_lang_operatingsystem_process_cpu_load ProcessCpuLoad (java.lang.C
pe=OperatingSystem><>ProcessCpuLoad)
# TYPE java_lang_operatingsystem_process_cpu_load untyped
java_lang_operatingsystem_process_cpu_load 0.0035742652899126287
# HELP java_lang_operatingsystem_system_cpu_load SystemCpuLoad (java.lang<type=OperatingSystem><>SystemCpuLoad)
# TYPE java_lang_operatingsystem_system_cpu_load untyped
java_lang_operatingsystem_system_cpu_load 0.1791104050833995
# HELP java_lang_operatingsystem_process_cpu_time ProcessCpuTime (java.lang<type=OperatingSystem><>ProcessCpuTime)
# TYPE java_lang_operatingsystem_process_cpu_time untyped
java_lang_operatingsystem_process_cpu_time 3.922E10
# HELP java_lang_threading_current_thread_cpu_time CurrentThreadCpuTime (java.lang<type=Threading><>CurrentThreadCpuTime)
# TYPE java_lang_threading_current_thread_cpu_time untyped
java_lang_threading_current_thread_cpu_time 1.160761966E9
100 133k 100 133k      0      129k      0:00:01 0:00:01 --:--:-- 129k
# HELP java_lang_threading_thread_cpu_time_enabled ThreadCpuTimeEnabled (java.lang<type=Threading><>ThreadCpuTimeEnabled)
# TYPE java_lang_threading_thread_cpu_time_enabled untyped
java_lang_threading_thread_cpu_time_enabled 1.0
# HELP java_lang_threading_thread_cpu_time_supported ThreadCpuTimeSupported (java.lang<type=Threading><>ThreadCpuTimeSupported)
# TYPE java_lang_threading_thread_cpu_time_supported untyped
java_lang_threading_thread_cpu_time_supported 1.0
# HELP java_lang_threading_current_thread_cpu_time_supported CurrentThreadCpuTimeSupported (java.lang<type=Threading><>CurrentThreadCpuTimeSupported)
# TYPE java_lang_threading_current_thread_cpu_time_supported untyped
java_lang_threading_current_thread_cpu_time_supported 1.0
# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
```

Vamos a crear un gráfico con la métrica que muestra el porcentaje de uso del CPU a nivel del sistema con `java_lang_operatingsystem_systemcpuload`

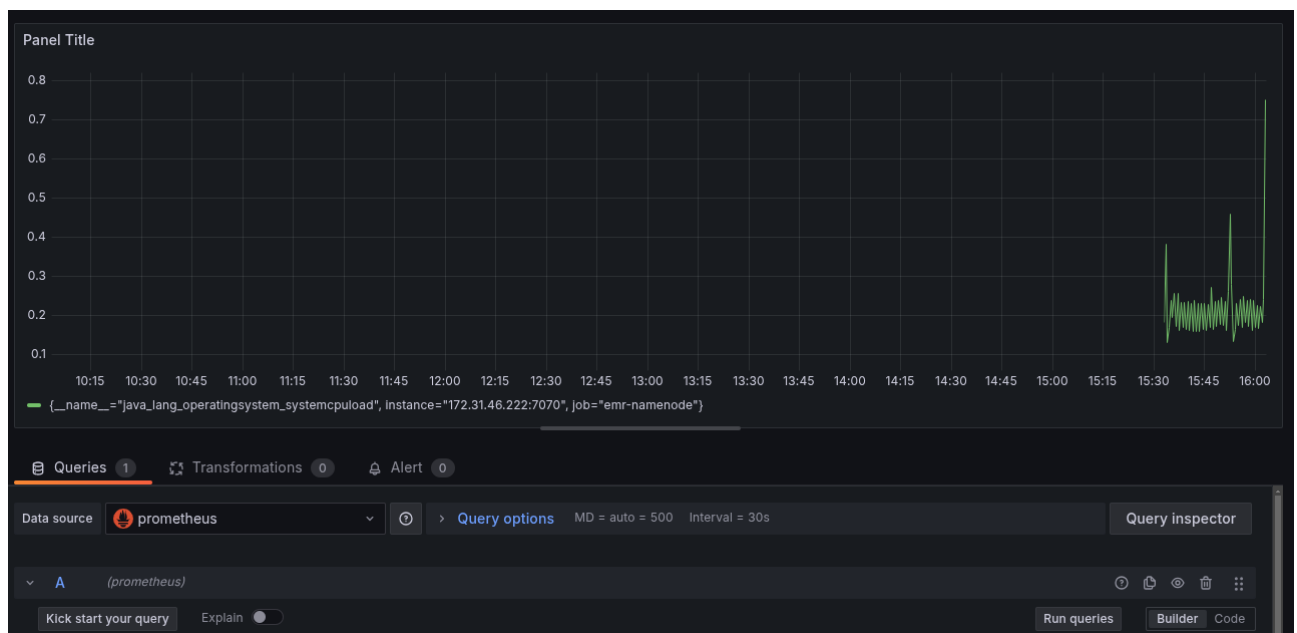
Una vez que hayamos indicado que queremos añadir una nueva visualización, se abrirá la siguiente pantalla



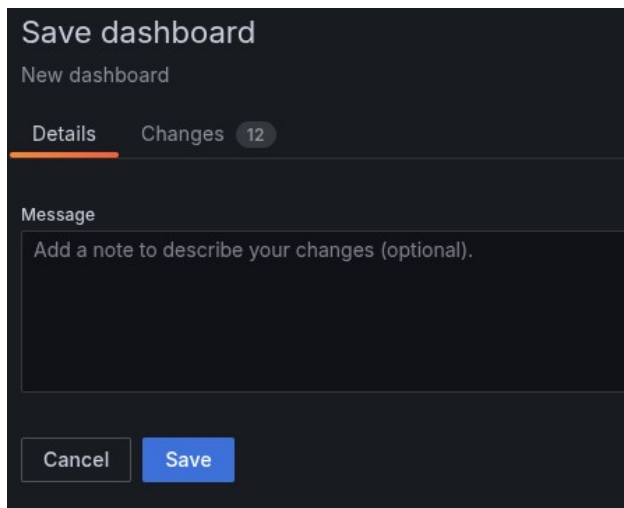
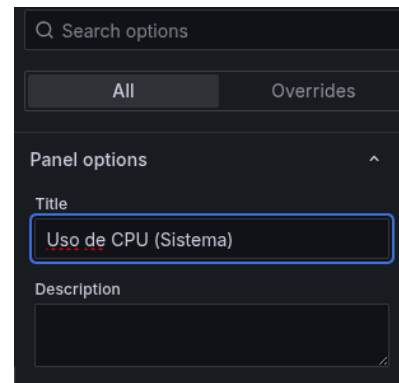
Indicaremos la métrica en el campo correspondiente



Y tras hacer click en el botón 'Run queries' se mostrará el gráfico

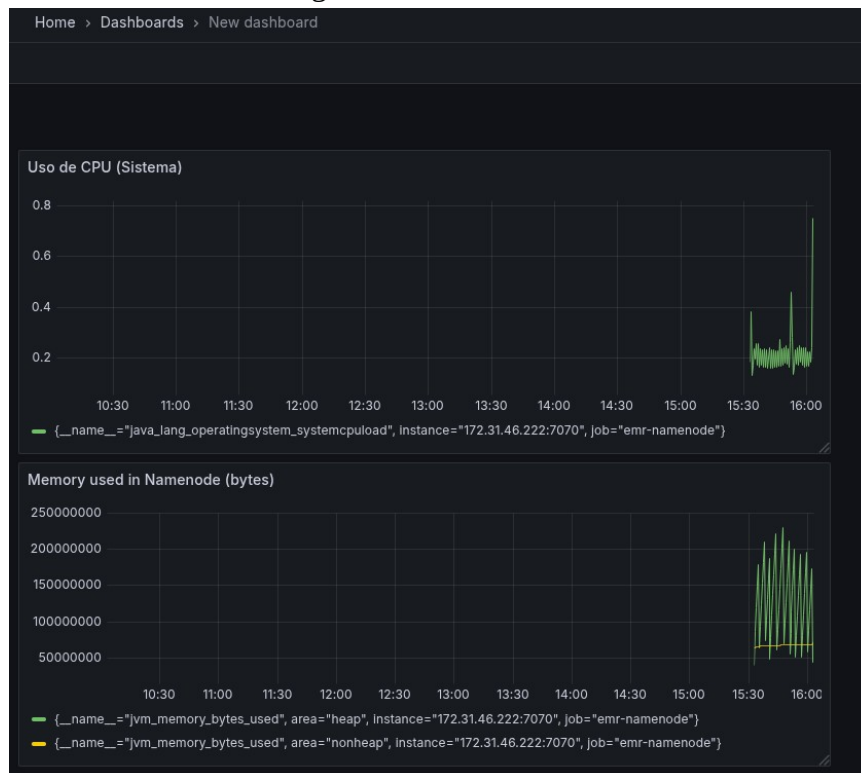


Ahora para modificar el nombre del panel, modificaremos el apartado de *Title* que aparece en el menú lateral de la derecha



Para guardarlo solo deberemos hacer click en ‘*Save dashboard*’ que aparece arriba a la derecha, es un botón de color azul, este desplegará un Panel y seleccionaremos *Save*.

Ahora se verán los dos gráficos en nuestro dashboard



```
jpop@jpop-03: /Documents/LIAD/BDA/UTG$ MONITORED/Evalables/Monitoring_AWS_EH$ curl http://52.90.182.109:7070/metrics | grep hdfs
```

% Total	% Received	% Xferd	Average Speed	Time Dload	Time Upload	Time Total	Time Spent	Time Left	Current Speed
0	0	0	0	0	0	0	0	0	0
# HELP hadoop_namenode_storageTypeStats_capacityused org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>capacityUsed}									
# HELP hadoop_namenode_storageTypeStats_capacityNonDisfused org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>capacityNonDisfused}									
# HELP hadoop_namenode_storageTypeStats_blockPoolused org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>blockPoolUsed}									
# HELP hadoop_namenode_storageTypeStats_capacityTotal org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>capacityTotal}									
# HELP hadoop_namenode_storageTypeStats_nodesInService org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>nodsInService}									
80	133k	80	107k	0	123k	0	0:00:01	--:--:--	0:00:01 123k# HELP hadoop_namenode_storageTypeStats_capacityremaining org.apache.hadoop.hdfs.server.blockmanagement.StorageTypeStats {Hadoop<service=NameNode, name=BlockStats, key=DISK><StorageTypeStats>capacityRemaining}
100	133k	100	133k	0	153k	0	--:--:--	--:~:~:~	153k

Table view

Last 6 hours

Refresh

Time series

Time

{__name__="hadoop_namenode_storage_typedstats_capacityused", instance="172.31.41.100"}

2025-03-29 15:33:00	22659072
2025-03-29 15:33:30	22659072
2025-03-29 15:34:00	22659072
2025-03-29 15:34:30	22659072
2025-03-29 15:35:00	22659072
2025-03-29 15:35:30	22659072
2025-03-29 15:36:00	22659072
2025-03-29 15:36:30	22659072
2025-03-29 15:37:00	22659072

Queries 1

Transformations 0

Alert 0

Data source

prometheus

Query options

MD = auto = 500 Interval = 30s

Query inspector

Click start your query

Explain

Run queries

Builder

Code

Metric

hadoop_namenode_storage_typedstats_capacityused

Label filters

Select label

=

Select value

x

+

Panel options

Title

HDFS used (bytes)

Description

Transparent background

Panel links

Repeat options

Tooltip

Tooltip mode

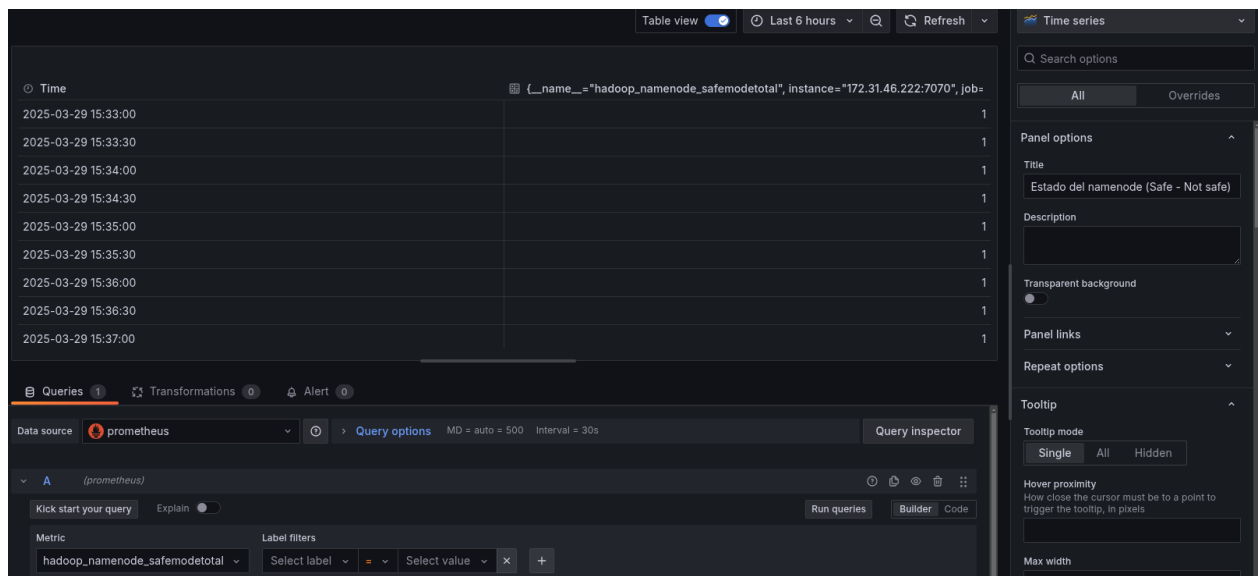
Single All Hidden

Hover proximity

How close the cursor must be to a point to trigger the tooltip, in pixels

Max width

La métrica a usar es `hadoop_namenode_safemode_total`



4. Resultado final del dashboard con las métricas

